Visual Recognition using DL HW2 Report

Author: 司徒立中 (111550159) | GitHub Link

Introduction

In this task, the dataset is a subset from Street View House Numbers (SVHN), consisting of 30,062 training images, 3,340 validation images, and 13,068 test images. Each image contains bounding boxes and corresponding labels for every digit present. The task is divided into two parts: the first involves detecting the class and bounding box of each digit in the image, while the second aims to predict the complete number represented by the detected digits. Performance is evaluated using mean Average Precision (mAP) for the first task and accuracy for the second.

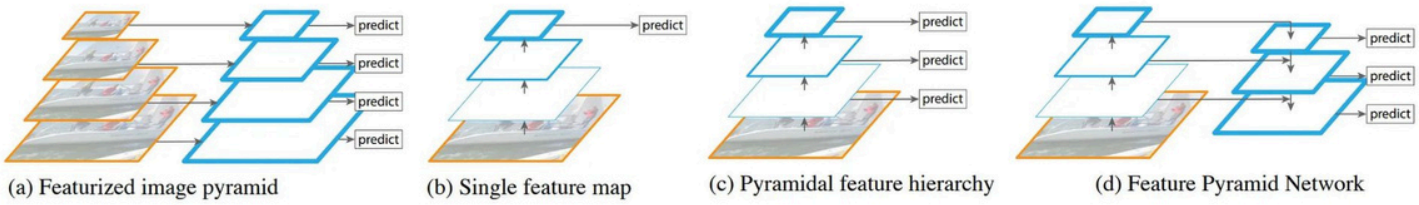
In addition, the strategies for this task are subject to certain constraints. There are two main restrictions as follows: First, no external data is allowed, in order to ensure that the focus of this assignment is on model architecture design rather than dataset collection. Second, only the Faster R-CNN [3] model is permitted for use in this task. Nevertheless, it's acceptable to modify its backbone, neck (Region Proposal Network), and head.

Method

	Seed	Epoch	Patience	Batch	Base LR	Other LR	Weight Decay	Scheduler
hyper-params	111550159	30	-	4	3e-5	3e-4	1e-5	Cosine Annealing

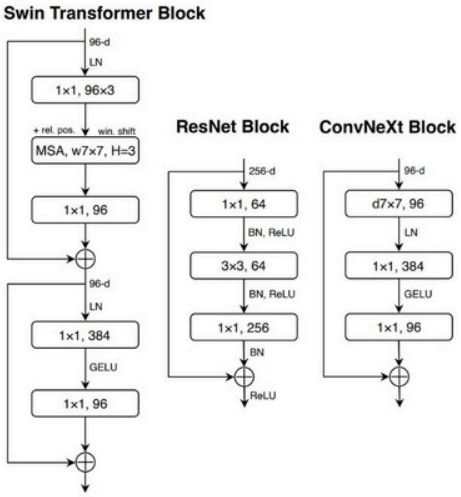
In this work, the methods focus on feature extraction through backbone modification. In my implementation, I utilized Feature Pyramid Networks (FPN) to fuse multi-scale features and used

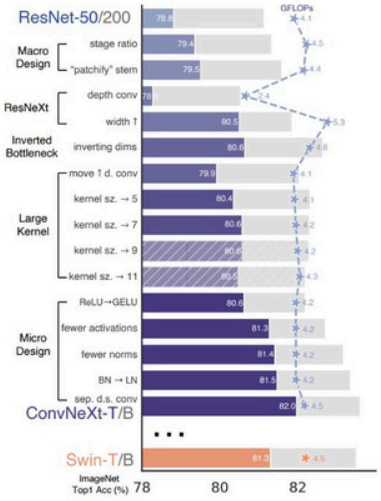
ConvNeXt as the model backbone. Some hyperparameters are shown above, such as epoch, batch size, and learning rate.



(a) Featurized image pyramid (b) Single feature map (c) Pyramidal feature hierarchy (d) Feature Pyramid Network

To address the challenge of scale variation, since digits can appear in different sizes, Feature Pyramid Networks (FPN) employ a top-down architecture with lateral connections. This approach merges semantically rich, low-resolution features with semantically weaker, high-resolution features, producing feature maps that are both semantically informative and spatially precise. Implementing this strategy significantly outperforms the strong baseline (0.35 | 0.70) for this assignment.

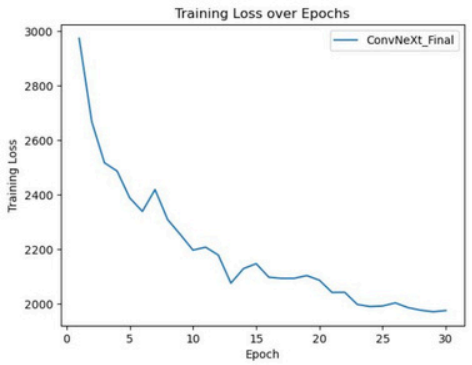


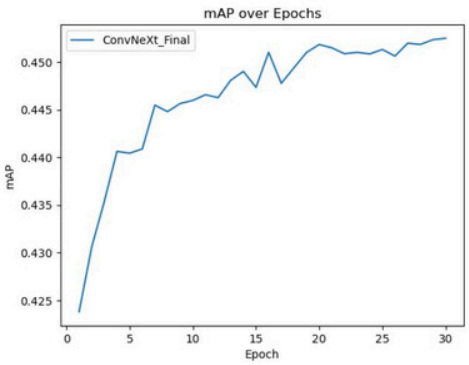


Additionally, compared to traditional backbones for Faster R-CNN such as ResNet [4], more advanced models like ConvNeXt [5] offer significantly better feature extraction capabilities. ConvNeXt [5] employs an inverted bottleneck structure (d7x7, 96), expanding the number of channels to improve feature extraction. Besides, it replaces the traditional ReLU with GELU, offering smoother activation, and substitutes BatchNorm with LayerNorm to enhance training stability. Therefore, I chose ConvNeXt [5] as the backbone and manually integrated it with FPN [7]. For more implementation details and performance comparisons, please refer to my 'Additional Experiments' section below. In short, the performance improved by replacing ResNet-50 [4] with ConvNeXt [5].

Finally, to improve the performance in Task 2, it is essential to set an appropriate threshold to combine a series of digits into a number. In my case, I used the results from Task 1 with a threshold of 0.5 to combine all digits based on their X coordinates. When the threshold was increased to 0.7, the accuracy improved by approximately 2%, which is a significant improvement.

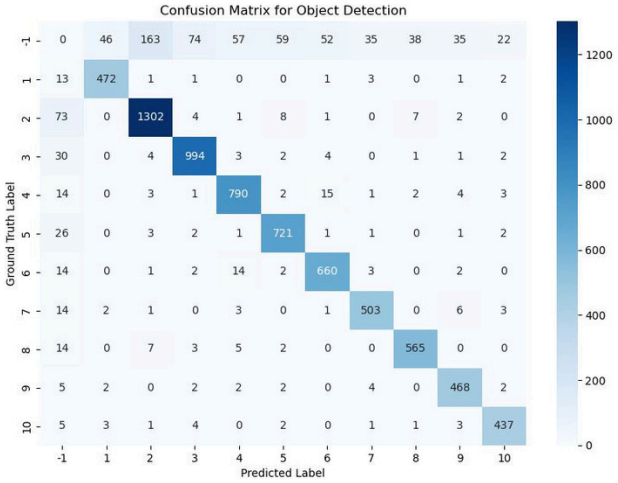
Results





Evaluation	Task1 (mAP)	Task2 (Acc)
Validation	0.4525	-
Test (Public)	0.4065	0.8597
Test (Private)	0.4091	0.8604

These two line charts show the training loss and validation mAP. As training progresses, the left chart demonstrates a gradual decrease in loss, indicating that the model is learning effectively over the epochs. On the right chart, the validation mAP consistently increases, suggesting that the model is generalizing well and capturing more relevant features. By the final epoch, the validation mAP reaches approximately 0.45, which aligns with the table on the right, where the model achieves 0.4525 on the validation set and around 0.41 on both the public and private test sets. For Task 2, the model's accuracy hovers around 0.86 on the test splits, showing that it maintains strong performance across different subsets of data.



The confusion matrix shows a clear pattern where most of the correct predictions fall along the diagonal, indicating that the model generally identifies objects accurately. However, there are several off-diagonal elements that reveal areas of confusion, where the model tends to misclassify one category as another. In particular, the presence of notable counts in the cells associated with the negative label "-1" suggests that the model sometimes detects objects that are not present in the ground truth, or fails to detect objects that should be there, leading to false positives and false negatives.

References

[1] (CVPR'14) Rich feature hierarchies for accurate object detection and semantic segmentation

[2] (ICCV'15) Fast R-CNN

[3] (NIPS'15) Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

[4] (CVPR'16) Deep Residual Learning for Image Recognition

[5] (CVPR'22) A ConvNet for the 2020s

[6] (Arxiv'23) ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders

[7] (CVPR'17) Feature Pyramid Networks for Object Detection

[8] (CVPR'18) Path Aggregation Network for Instance Segmentation

[9] (ICCV'19) CARAFE: Content-Aware ReAssembly of FEatures

[10] (ICPR'20) A novel Region of Interest Extraction Layer for Instance Segmentation

[11] (CVPR'21) Dynamic Head: Unifying Object Detection Heads with Attentions

[12] (NIPS'20) Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection

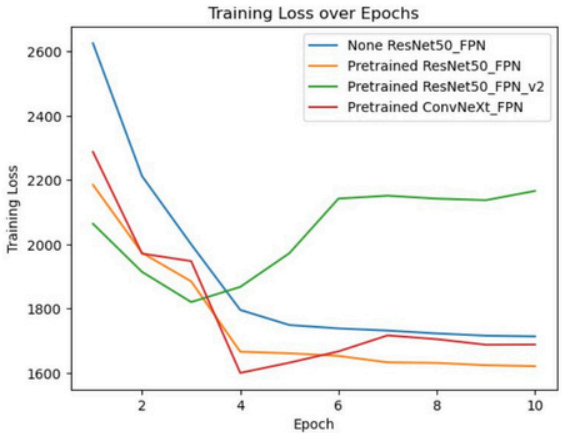
[13] Pytorch - Models and pre-trained weights

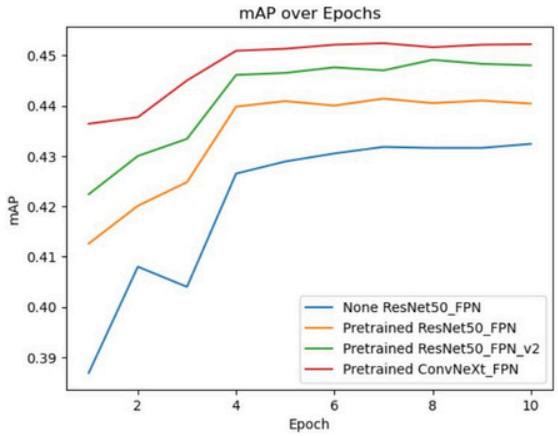
[14] Hugging Face - Timm - convnextv2_base.fcmae

[15] Pytorch - Albumentations

[16] MMdetection - GitHub Implamentation

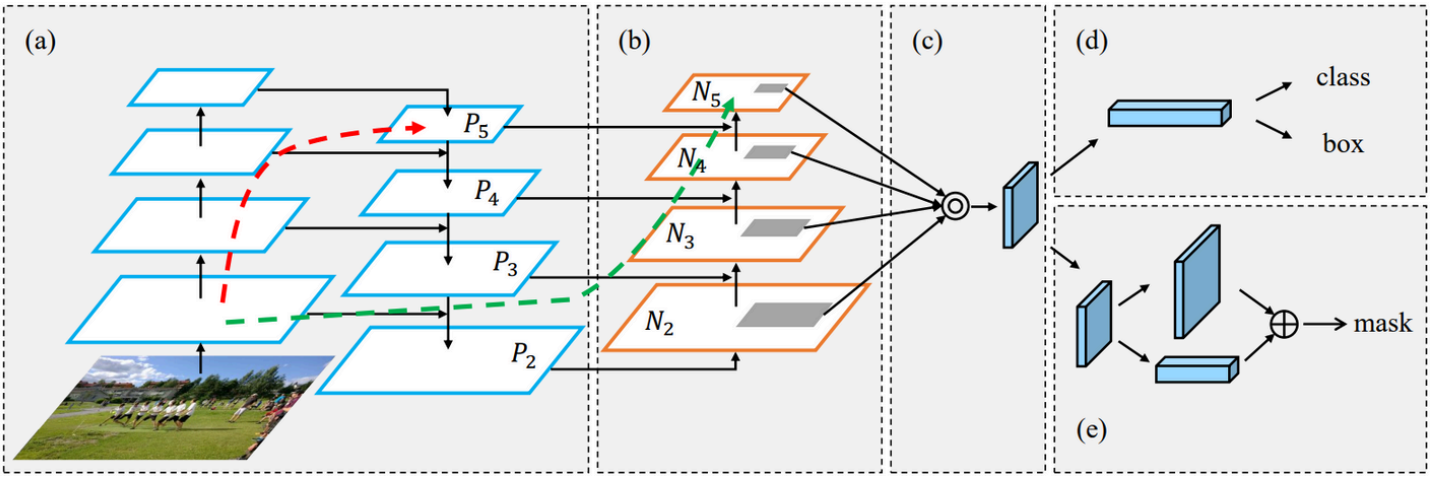
Additional experiments



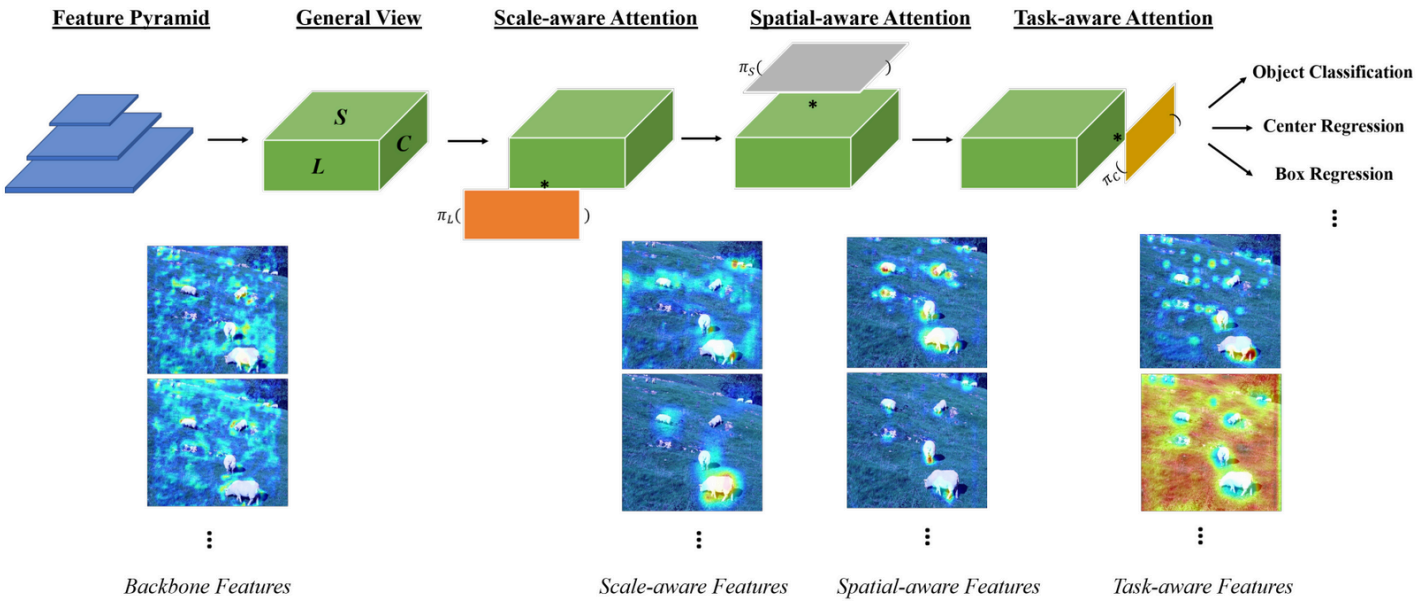


At the beginning of experiments, I attempted to apply various backbone of Faster R-CNN for feature extraction. The left chart shows the training loss for each backbone, while the right chart illustrates each backbone's mAP score on Task 1. Although some advanced backbones have more changes in training loss compared to the vanilla ResNet50_FPN, they usually achieve a higher final validation mAP score. This outcome implies that while advanced architectures may have more complex parameter spaces leading to greater training instability, they can ultimately achieve better performance in terms of accuracy, providing valuable insights for backbone selection.

In the implementation of ConvNeXt_FPN, the ConvNeXt backbone was pre-trained on the ImageNet dataset because the COCO dataset was not available through torchvision. Additionally, I selected layers "feature1, 3, 5, 7" as the inputs for the FPN. This choice was made as these layers offer a good balance between low-level details and high-level semantic information, which helps in building robust feature representations for the task.



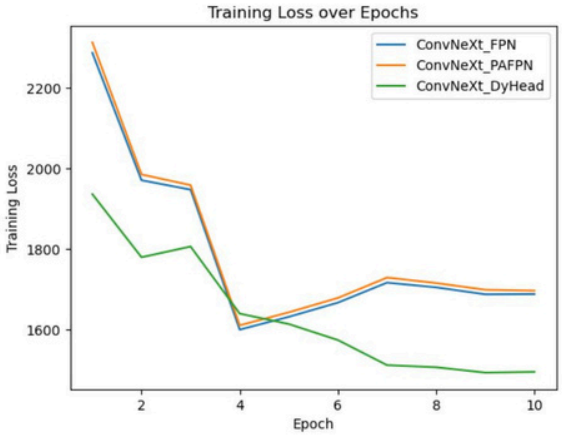
(Architecture of PAFPN [8])

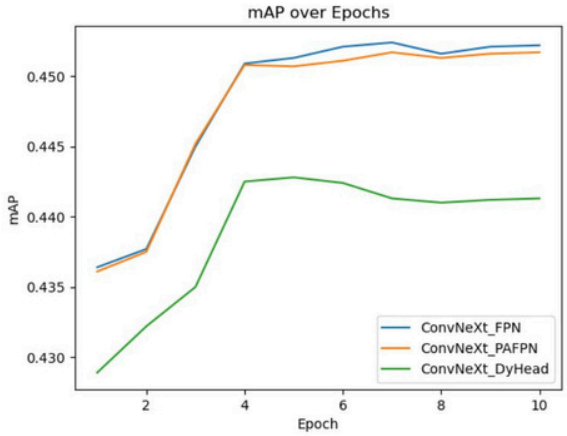


(Architecture of DyHead [11])

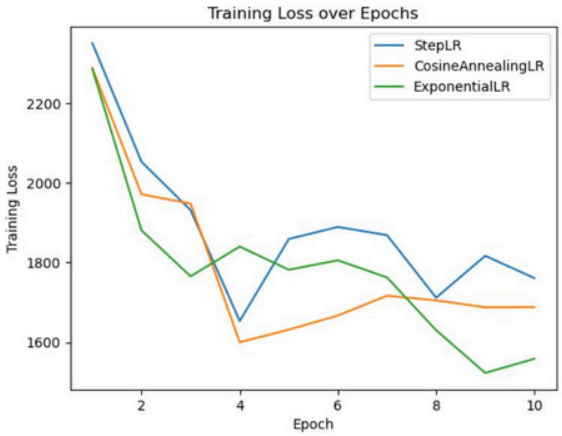
Rather than directly applying FPN as the final extraction layer, I have experimented with alternative approaches such as PAFPN [8] and DyHead [11] for further feature extraction. These methods aim to enhance the representation of multi-scale information and improve the overall performance of the model.

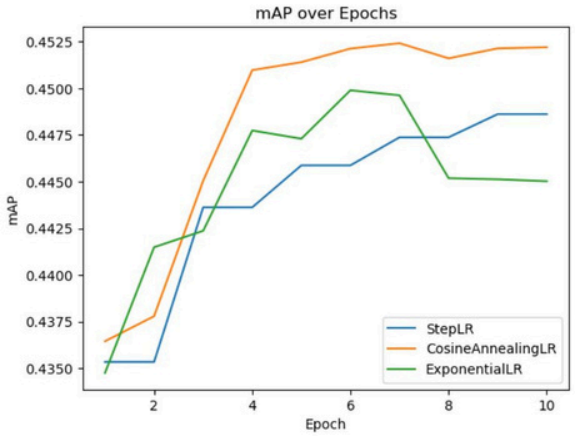
Instead of only using a top-down pathway to merge high-level semantic features with low-level details, PAFPN [8] adds a bottom-up path. This extra pathway helps better combine spatial details from the lower layers with richer semantic information from the higher layers. Meanwhile, DyHead [11] applies attention mechanisms across different dimensions: it captures scale-aware information by focusing on features at various resolutions, incorporates spatial-aware attention to emphasize important regions within the feature maps, and utilizes task-aware attention to adaptively weigh features relevant to specific tasks like classification or localization.



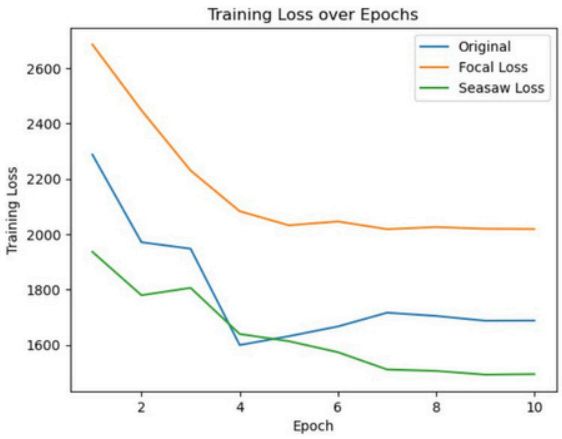


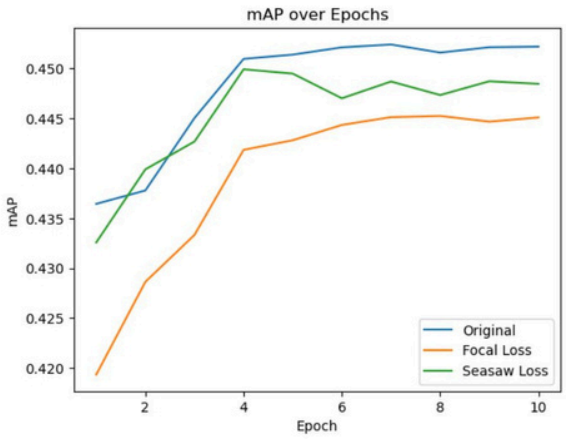
The left chart shows the training loss for each strategy, while the right chart illustrates each strategy's mAP score on Task 1. Theoretically, the performances of PAFPN [8] and DyHead [11] should be much better than original FPN. As seen in the training curves, DyHead tends to reduce the loss the fastest, whereas PAFPN provides a more balanced descent. On the mAP chart, each approach reaches comparable final scores, but their convergence patterns differ slightly. To ensure the original model's stability and reliability, it is reasonable to select the most classic approach FPN.





Next, I tried different learning schedulers to aid convergence. The left chart displays the training loss when using StepLR, CosineAnnealingLR, and ExponentialLR, while the right chart shows the corresponding mAP for each scheduler. StepLR lowers the learning rate at fixed intervals, CosineAnnealingLR gradually reduces it following a cosine function, and ExponentialLR scales it down by a constant factor after every epoch. As seen in the graphs, each method influences how quickly and smoothly the model converges, with subtle differences in the final mAP. Since selecting an optimal learning scheduler can help strike a balance between steady training and a high final accuracy, I chose CosineAnnealingLR as my final learning rate scheduler.





Finally, I tried using different loss functions. In the original setup, Faster R-CNN follows torchvision's default configuration, which applies cross-entropy for classification along with three other losses (e.g., bounding box regression). I then replaced the classification component with Focal Loss and Seesaw Loss, respectively, to address potential class imbalance more effectively.

The left chart shows the changes in training loss under each loss function, while the right chart illustrates the corresponding mAP. As the results suggest, replacing the original classification loss with Focal Loss or Seesaw Loss affects both convergence behavior and detection performance. While Seesaw Loss achieves the lowest training loss and Focal Loss is designed to tackle class imbalance, neither consistently outperformed the original setup in terms of validation mAP. Therefore, to achieve the best overall performance, I decided to retain the default loss functions provided in torchvision's Faster R-CNN implementation.

Code Reliability

```
(env) cookies@gpu8:~/VRDL$ flake8 dataset.py
(env) cookies@gpu8:~/VRDL$ flake8 evaluate.py
(env) cookies@gpu8:~/VRDL$ flake8 infer.py
(env) cookies@gpu8:~/VRDL$ flake8 main.py
(env) cookies@gpu8:~/VRDL$ flake8 models.py
(env) cookies@gpu8:~/VRDL$ flake8 train.py
(env) cookies@gpu8:~/VRDL$ flake8 utils.py
(env) cookies@gpu8:~/VRDL$
```